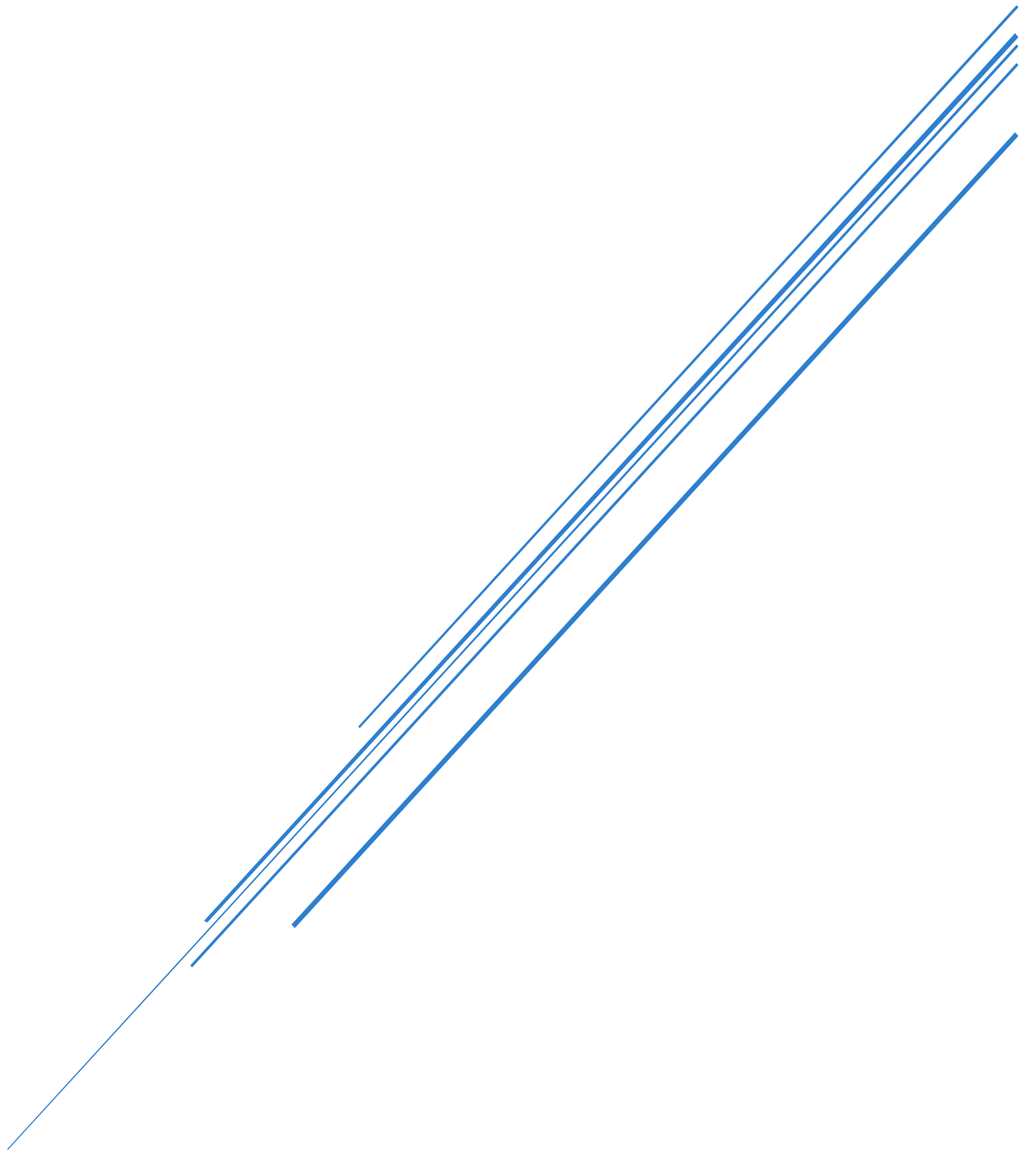


BACKEND LIGERITO



Diego Fernández López

Contenido

1- Definición del proyecto	2
Elección de la idea y alcance.	2
Identificación del problema que resuelve	2
Objetivos generales y específicos	3
Tecnologías a utilizar:	4
2- Análisis de requisitos.....	4
Requisitos Funcionales.	4
Requisitos no funcionales.	6
Identificación de usuarios y roles.	7
Casos de uso principales	7
3- Arquitectura de la aplicación.....	9
Descripción de cada capa	11
4- Descripción de entidades y relaciones JPA	12
Descripción de servicios y controladores	14
5. Manual técnico.....	16
6. Informe de pruebas.....	17

1-Definición del proyecto

Elección de la idea y alcance.

Como apasionado de la montaña, una de las tareas críticas previas a cualquier actividad es la preparación del equipo. Mientras que para una salida de un día la planificación es sencilla, en actividades de varios días o en disciplinas de ultra distancia, la **gestión del peso y el material** se vuelve vital para el éxito y la seguridad.

Ligerito nace para solucionar esta necesidad, permitiendo a los usuarios gestionar su inventario de artículos deportivos y organizar mochilas específicas para cada actividad. La aplicación ofrece una visión detallada del **peso total y el desglose por categorías**, optimizando la carga de forma visual y eficiente. Además, fomenta el aspecto social permitiendo compartir configuraciones de material entre miembros de la comunidad.

En cuanto al **alcance**, el sistema se centra en una versión funcional que cubre:

- **Gestión de usuarios:** Registro e inicio de sesión.
- **Armario virtual:** Inventario general de artículos personales.
- **Configurador de mochilas:** Creación y organización de listas de equipo por actividad.
- **Categorización inteligente:** Clasificación de objetos (textil, pernocta, alimentación, etc.).
- **Cálculo métrico:** Monitorización automática de pesos totales y parciales.

Identificación del problema que resuelve

El problema central que aborda **Ligerito** es la falta de herramientas optimizadas para el control y análisis del peso del equipo en actividades de montaña y larga distancia o resistencia. Actualmente, los deportistas suelen recurrir a métodos manuales que presentan ineficiencias críticas:

- **Registro manual o analógico:** El pesado individual y la suma en papel o notas digitales es un proceso propenso a errores de cálculo, tedioso y difícil de replicar en futuras salidas.
- **Enfoque reactivo:** Pesar la mochila una vez terminada obliga a vaciarla y reorganizarla si se excede el peso deseado, lo que supone una pérdida de tiempo y una planificación ineficiente.

- **Dispersión de datos:** La ausencia de un inventario centralizado obliga al usuario a pesar el mismo artículo (piolet, hornillo, saco) en repetidas ocasiones para diferentes actividades.

La Solución: Ligerito

Ligerito transforma este proceso en una solución estructurada y digital. Al centralizar un **inventario técnico reutilizable**, el usuario elimina la redundancia de datos. La aplicación permite una planificación **proactiva**, donde el peso se gestiona de forma virtual antes de siquiera tocar la mochila físicamente, garantizando precisión, ahorro de tiempo y una distribución de cargas optimizada por categorías.

Objetivos generales y específicos

Objetivo general

Desarrollar una aplicación web que permita a los usuarios organizar su equipaje de forma estructurada, reutilizar objetos entre distintas mochilas y controlar el peso total y por categorías de manera sencilla e intuitiva.

Objetivos específicos

- Diseñar un modelo de datos que represente usuarios, mochilas, categorías y objetos reutilizables.
- Permitir el registro e inicio de sesión de usuarios.
- Implementar un armario general de objetos asociado a cada usuario.
- Permitir la creación de mochilas o listas de equipaje personalizadas.
- Permitir la clasificación de objetos dentro de categorías.
- Facilitar la reutilización de un mismo objeto en varias mochilas.
- Calcular automáticamente el peso total de una mochila y su distribución por categorías.
- Publicar la mochila o lista de viaje que se decida permitiendo la consulta de otros usuarios.

Tecnologías a utilizar:

Para el desarrollo del proyecto se utilizarán tecnologías diferenciadas para frontend y backend. En la parte servidor se empleará **Spring Boot** con **Spring Data JPA/Hibernate** para la persistencia de datos, junto con **MySQL** como sistema gestor de base de datos. En la parte cliente se utilizará **React** con **Vite** para la construcción de la interfaz y **Tailwind CSS** para el diseño visual. Además, se emplearán herramientas complementarias como **Postman** para la validación de endpoints, **Docker** para la ejecución local de la base de datos y **Visual Studio Code** como entorno de desarrollo.

2- Análisis de requisitos

Requisitos Funcionales.

Los requisitos se han reestructurado para reflejar un sistema de inventario dinámico y persistente, priorizados según su impacto en la experiencia del usuario.

1. Gestión de Usuarios y Acceso (Prioridad Crítica)

- **RF-01. Registro de usuario:** El sistema permitirá crear cuentas únicas vinculadas a un correo electrónico.
- **RF-02. Autenticación (Login):** Acceso seguro al ecosistema personal (mochilas, armario y categorías).
- **RF-03. Cierre de sesión (Logout):** Finalización de sesión para garantizar la privacidad de los datos.

2. Gestión de Mochilas y Control de Peso (Prioridad Alta)

- **RF-04. Creación de listas:** El usuario podrá generar múltiples mochilas con nombres personalizados.
- **RF-05. Cálculo de peso total (CRÍTICO):** El sistema automatiza el cálculo del peso total en tiempo real:

$$Peso_{Total} = \sum_{i=1}^n (Peso_{item_i} \times Cantidad_i)$$

Desglose de la fórmula

La ecuación se lee como: "El Peso Total es la suma de los pesos individuales multiplicados por su cantidad".

- **$Peso_{Total}$** : Es el resultado final (el peso de la mochila completa o de una categoría específica).
 - $\sum_{i=1}^n$: Este es el símbolo de **Sumatorio**. Indica que vamos a sumar una lista de elementos, empezando por el primero ($i=1$) hasta llegar al último (n). En tu caso, n es el número total de artículos diferentes que has metido en la mochila.
 - **$Peso_{item_i}$** : Es el peso unitario del artículo número i . Por ejemplo, si el artículo 1 es una "Barrita energética", su peso unitario sería 50g.
 - **$Cantidad_i$** : Es el número de unidades que llevas de ese artículo específico. Siguiendo el ejemplo, si llevas 5 barritas, la cantidad es 5.
 - **$(Peso_{item_i} \times Cantidad_i)$** : Es el peso parcial por cada tipo de artículo.
- **RF-06. Desglose de peso por categorías:** Visualización del peso acumulado por cada categoría creada por el usuario.
 - **RF-07. Gestión de visibilidad de la mochila:** El usuario podrá cambiar el estado de una mochila entre pública y privada. Cuando una mochila se marque como pública, aparecerá en la sección de exploración para que otros usuarios puedan visitarla y consultarla en modo lectura.
 - **RF-08. Eliminación definitiva:** El usuario podrá borrar cualquier mochila, incluida la última, previa confirmación de seguridad. Si no existen mochilas, el sistema permanecerá en estado vacío hasta la creación de una nueva.

3. Gestión de Inventario y Personalización (Prioridad Media)

- **RF-09. Inventario Maestro (Armario):** El sistema mantendrá un inventario personal por usuario donde se registrarán todos los objetos reutilizables creados, permitiendo su uso en diferentes mochilas sin duplicar datos.
- **RF-10. Categorías personalizadas por mochila:** El sistema permitirá al usuario crear, nombrar y gestionar categorías propias dentro de cada mochila para organizar el equipamiento y calcular el reparto de peso por secciones.
- **RF-11. Alta de ítems reutilizables con metadatos:** El usuario podrá crear objetos desde una mochila, indicando nombre, peso en gramos, descripción técnica opcional y enlace opcional. Una vez creados, estos pasarán al armario personal para poder reutilizarse en otras mochilas sin duplicar datos.
- **RF-12. Control dinámico de unidades:** El usuario podrá incrementar o decrementar la cantidad de un objeto dentro de una mochila,

actualizándose automáticamente el contenido de la lista y el peso total asociado.

- **RF-13. Sincronización de datos globales del objeto:** El sistema permitirá modificar el peso, la descripción y, en su caso, otros metadatos globales de un objeto reutilizable desde cualquier mochila donde aparezca, propagando el cambio al armario general y al resto de mochilas que reutilicen dicho objeto.

4. Visualización y Comunidad (Prioridad Baja)

- **RF-14. Exploración de comunidad:** Acceso en modo lectura a las mochilas marcadas como públicas por otros usuarios para servir de inspiración o guía.

Requisitos no funcionales.

- **RNF-01. Usabilidad:** La aplicación deberá ofrecer una interfaz clara, sencilla e intuitiva, de forma que el usuario pueda registrarse, iniciar sesión, gestionar mochilas y consultar su armario sin necesidad de formación previa.
- **RNF-02. Persistencia de datos:** El sistema deberá almacenar de forma persistente la información de usuarios, mochilas, categorías y objetos, evitando su pérdida al cerrar la aplicación o reiniciar el sistema.
- **RNF-03. Consistencia de datos:** El sistema deberá mantener la coherencia entre el armario general y las mochilas del usuario, garantizando que los cambios en los datos globales de un objeto se reflejen correctamente en todos los lugares donde se reutilice.
- **RNF-04. Seguridad básica:** El acceso a la información personal deberá estar restringido a usuarios autenticados, impidiendo el acceso o modificación de datos ajenos.
- **RNF-05. Mantenibilidad:** El proyecto deberá estar organizado de forma modular y estructurada, separando responsabilidades entre interfaz, lógica de negocio y persistencia, para facilitar su comprensión, corrección y ampliación futura.
- **RNF-06. Validación de entradas:** El sistema deberá validar los datos introducidos por el usuario en formularios y operaciones, comprobando que campos como email, contraseña, nombre, peso o cantidad tengan un formato y valor correctos antes de ser procesados.

Identificación de usuarios y roles.

Usuario estándar

Es el rol principal de la aplicación.

Sus funciones serían:

- registrarse e iniciar sesión
- gestionar su armario personal
- crear, editar y eliminar sus mochilas
- crear categorías dentro de sus mochilas
- añadir y reutilizar objetos
- consultar pesos totales y por categoría
- decidir si una mochila es pública o privada
- explorar mochilas públicas de otros usuarios en modo lectura

Casos de uso principales

CU-01. Registrarse en la aplicación

Actor principal: Usuario

Descripción: Un visitante crea una cuenta nueva introduciendo nick, correo electrónico y contraseña para poder acceder a su espacio personal.

CU-02. Iniciar sesión

Actor principal: Usuario

Descripción: Un usuario registrado accede a la aplicación mediante su correo electrónico y contraseña para gestionar su armario y sus mochilas.

CU-03. Cerrar sesión

Actor principal: Usuario

Descripción: El usuario finaliza su sesión para salir de su espacio personal y proteger su información.

CU-04. Crear una mochila

Actor principal: Usuario

Descripción: El usuario crea una nueva mochila o lista de equipaje asignándole un nombre personalizado.

CU-05. Gestionar categorías de una mochila

Actor principal: Usuario

Descripción: El usuario crea, organiza o elimina categorías dentro de una mochila para clasificar mejor sus objetos.

CU-06. Crear un objeto desde una mochila

Actor principal: Usuario

Descripción: El usuario crea un nuevo objeto indicando nombre, peso y datos opcionales como descripción o enlace. Una vez creado, el objeto pasa a formar parte del armario general.

CU-07. Añadir un objeto del armario a una mochila

Actor principal: Usuario

Descripción: El usuario selecciona un objeto ya existente en su armario y lo añade a una categoría de la mochila activa, reutilizando así material sin duplicar datos.

CU-08. Modificar cantidad de un objeto en una mochila

Actor principal: Usuario

Descripción: El usuario incrementa o reduce la cantidad de un objeto dentro de una mochila, actualizándose automáticamente el peso total.

CU-09. Editar datos globales de un objeto reutilizable

Actor principal: Usuario

Descripción: El usuario modifica atributos globales del objeto, como el peso o la descripción, propagándose el cambio al armario y a todas las mochilas donde dicho objeto aparezca.

CU-10. Eliminar un objeto de una mochila

Actor principal: Usuario

Descripción: El usuario elimina un objeto concreto de una mochila sin borrar necesariamente el objeto base del armario general.

CU-11. Consultar el peso total de una mochila

Actor principal: Usuario

Descripción: El sistema muestra automáticamente el peso total de la mochila teniendo en cuenta los objetos incluidos y sus cantidades.

CU-12. Consultar el desglose de peso por categorías

Actor principal: Usuario

Descripción: El usuario puede visualizar cuánto peso corresponde a cada categoría dentro de la mochila.

CU-13. Cambiar la visibilidad de una mochila

Actor principal: Usuario

Descripción: El usuario marca una mochila como pública o privada según quiera compartirla o mantenerla solo para uso personal.

CU-14. Explorar mochilas públicas

Actor principal: Usuario

Descripción: El usuario accede en modo lectura a mochilas públicas creadas por otros usuarios para inspirarse o consultar ejemplos.

CU-15. Consultar el detalle de una mochila pública

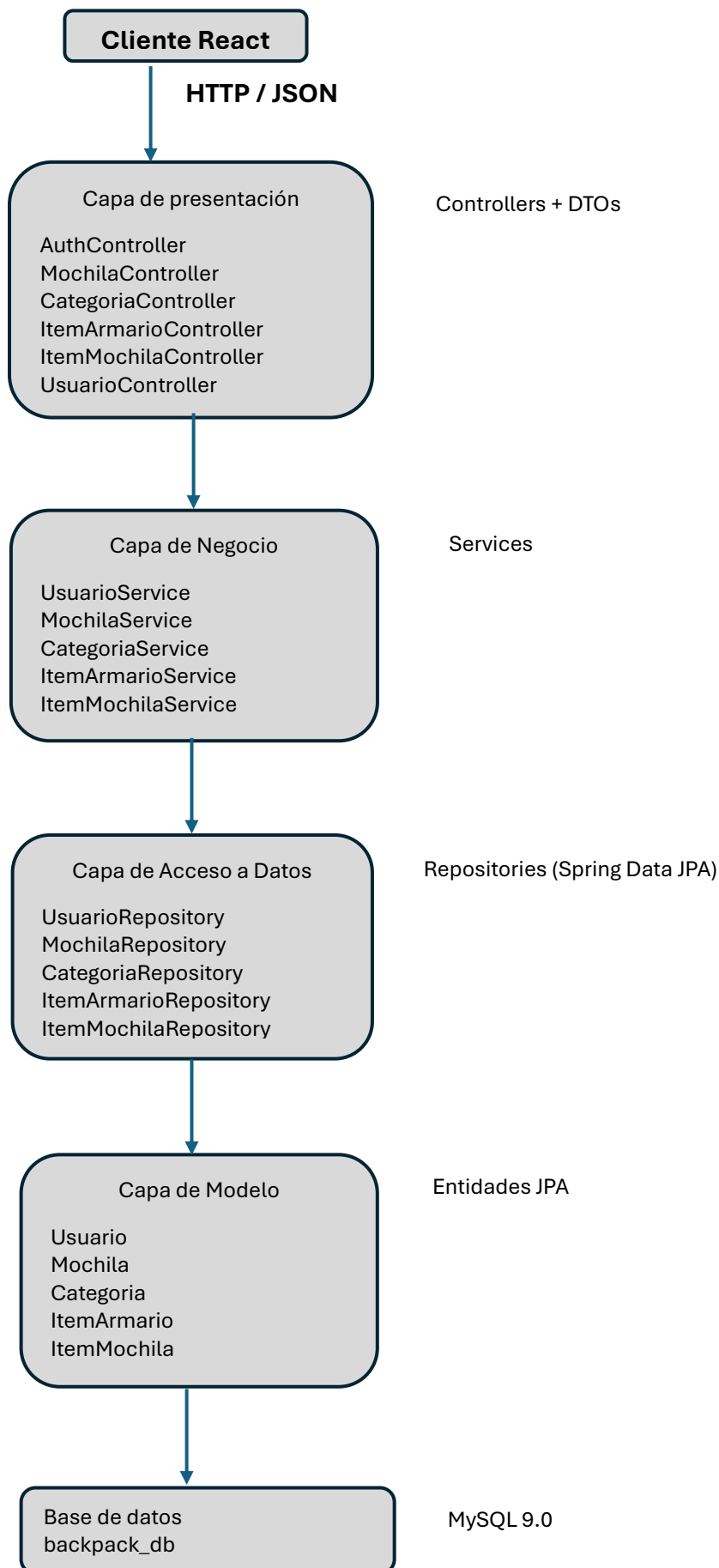
Actor principal: Usuario

Descripción: El usuario visualiza el contenido de una mochila pública ajena, incluyendo sus categorías y objetos, sin posibilidad de modificarla

3-Arquitectura de la aplicación.

El backend es una API REST desarrollada con Spring Boot que expone los datos en formato JSON, y el frontend es una Single Page Application (SPA) construida con React que consume esa API. Ambas partes se ejecutan de forma independiente y se comunican exclusivamente a través de HTTP.

Arquitectura en capas del backend:



Descripción de cada capa

Capa de presentación — Controladores y DTOs

Los controladores reciben las peticiones HTTP, delegan toda la lógica en los servicios y devuelven la respuesta. No contienen lógica de negocio. Para evitar exponer directamente las entidades JPA, se usan **DTOs** (*Data Transfer Objects*) diferenciados por operación: `CreateRequest` para crear, `PatchRequest` para actualizaciones parciales y `Response` para la salida.

Por ejemplo, cuando el frontend pide crear una mochila, el controlador recibe un `MochilaCreateRequest`, lo pasa al servicio, y devuelve un `MochilaResponse` — nunca la entidad `Mochila` directamente.

Capa de negocio — Servicios

Los servicios contienen todas las reglas de negocio. Algunos ejemplos concretos:

- `UsuarioService` valida que el email y el nick no estén duplicados antes de registrar un usuario, y usa `BCrypt` para comparar contraseñas en el login.
- `ItemMochilaService` implementa una lógica de acumulación inteligente: si se añade a una mochila un ítem que ya existe en la misma categoría, incrementa la cantidad en lugar de crear un duplicado.
- `MochilaService` calcula el peso total de una mochila sumando el peso de todos sus ítems multiplicado por su cantidad.

Capa de acceso a datos — Repositorios

Se usan interfaces que extienden `JpaRepository`, lo que proporciona las operaciones CRUD básicas sin necesidad de escribir SQL. Para consultas específicas del negocio se definen métodos con nombre derivado o con `@Query`. Por ejemplo, `MochilaRepository` tiene un método para obtener solo las mochilas públicas, y `CategoriaRepository` tiene una búsqueda por nombre insensible a mayúsculas para evitar categorías duplicadas como "Ropa" y "ropa".

Base de datos — MySQL

El esquema se gestiona con Hibernate en modo `update`: al arrancar la aplicación, Hibernate sincroniza las tablas con las entidades JPA automáticamente. Esto facilita el desarrollo iterativo sin necesidad de escribir scripts de migración.

Seguridad

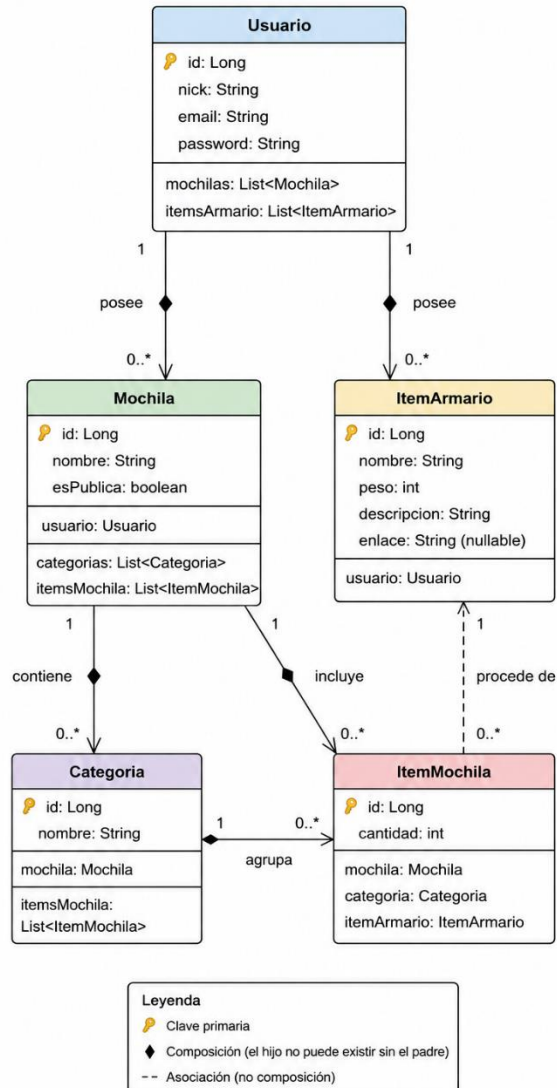
La seguridad está gestionada por **Spring Security** con las siguientes decisiones:

- Las contraseñas se almacenan hasheadas con **BCrypt**.

4-Descripción de entidades y relaciones JPA

Modelo de datos

El modelo de datos se compone de cinco entidades que representan el dominio completo de la aplicación:



Descripción de cada entidad

Usuario

Contiene nick, email y contraseña (almacenada con hash BCrypt). Posee dos colecciones propias: sus mochilas y los ítems de su armario personal. Ambas están configuradas con CascadeType.ALL y orphanRemoval = true, lo que significa que al eliminar un usuario se eliminan en cascada todas sus mochilas y todos sus ítems del armario automáticamente.

Mochila

Representa una lista de ítems de una actividad concreta. Tiene un nombre, un flag esPublica que determina si es visible para el resto de los usuarios, y pertenece a un Usuario mediante una relación @ManyToOne. A su vez, contiene colecciones de Categoria e ItemMochila, también con cascada y orphanRemoval.

Categoria

Agrupar los ítems dentro de una mochila (por ejemplo: "Ropa", "Dormir", "Alimentación"). Pertenece a una Mochila y contiene una colección de ItemMochila. Al eliminar una categoría, todos sus ítems asociados en la mochila se eliminan en cascada.

ItemArmario

Representa un ítem del inventario personal del usuario: nombre, peso en gramos, descripción técnica opcional y enlace de referencia o compra opcional. Es el objeto reutilizable base: no pertenece a ninguna mochila concreta, sino al usuario, lo que permite añadirlo a múltiples mochilas sin duplicar datos.

ItemMochila

Es la entidad central del modelo. Actúa como tabla de unión enriquecida entre Mochila, Categoria e ItemArmario, añadiendo el campo cantidad. Este diseño es lo que permite la reutilización: el mismo ItemArmario (el artículo real del armario) puede aparecer en varias mochilas o en varias categorías de la misma mochila, cada instancia con su propia cantidad. Todas sus relaciones son @ManyToOne con carga lazy para evitar consultas innecesarias.

Decisiones de diseño del modelo

Decisión	Justificación
CascadeType.ALL + orphanRemoval en Usuario y Mochila	Garantiza integridad referencial: borrar un usuario o una mochila limpia toda su jerarquía automáticamente
FetchType.LAZY en todas las relaciones @ManyToOne	Evita cargar datos innecesarios en cada consulta; los datos se cargan solo cuando se accede a ellos
@JsonIgnore en todas las colecciones @OneToMany	Previene bucles infinitos de serialización JSON al exponer las entidades (por ejemplo, Usuario → Mochila → Usuario)
ItemMochila como entidad propia en lugar de join table simple	Necesario porque la relación lleva datos propios (cantidad), lo que convierte la asociación en una entidad de pleno derecho

Descripción de servicios y controladores

Servicios

Los servicios son la capa donde reside toda la lógica de negocio. Los controladores les delegan el trabajo y ellos son los únicos que hablan con los repositorios.

UsuarioService

Gestiona el registro y la autenticación de usuarios. En el registro comprueba antes de guardar que ni el email ni el nick estén ya en uso, lanzando excepciones personalizadas EmailDuplicadoException o NickDuplicadoException en caso contrario. La contraseña nunca se guarda en texto plano: se codifica con BCrypt antes de persistirla. En el login, recupera el usuario por email y usa passwordEncoder.matches() para comparar la contraseña introducida con el hash almacenado, devolviendo un 401 si las credenciales no son correctas.

MochilaService

Gestiona el ciclo de vida completo de las mochilas. Además del CRUD estándar, implementa el cálculo del peso total mediante un stream sobre los ítems de la mochila:

```
int pesoTotal = mochila.getItemsMochila().stream()
    .mapToInt(item -> item.getItemArmario().getPeso() * item.getCantidad())
    .sum();
```

También expone las mochilas públicas: listarPublicas() devuelve todas las marcadas como esPublica = true junto con el nick del propietario y el peso total

calculado, y obtenerDetallePublic() devuelve el detalle completo con categorías e ítems, comprobando primero que la mochila exista y sea pública.

CategoriaService

Gestiona las categorías dentro de cada mochila. Antes de crear una categoría comprueba que no exista ya otra con el mismo nombre en la misma mochila (búsqueda insensible a mayúsculas), devolviendo un 409 en caso de duplicado. Esto evita tener, por ejemplo, "Textil" y "textil" como categorías distintas.

ItemArmarioService

Gestiona el inventario personal del usuario. Implementa creación, listado por usuario, eliminación y actualización parcial. En el PATCH solo se modifican los campos que llegan no nulos en el request (peso, descripción, enlace), y valida que el peso no sea negativo antes de aplicar el cambio.

ItemMochilaService

Contiene la lógica más relevante del dominio. Al añadir un ítem a una mochila, primero valida que la mochila, la categoría y el ítem de armario existan, y que la categoría pertenezca efectivamente a esa mochila. Después busca si ya existe esa combinación exacta (itemArmario + mochila + categoria): si existe, incrementa la cantidad en uno; si no, crea un registro nuevo con cantidad 1. Esto implementa la reutilización sin duplicar datos.

Controladores y endpoints

Los controladores reciben las peticiones HTTP, validan el body con @Valid cuando corresponde, delegan en el servicio y devuelven la respuesta con el código HTTP adecuado. No contienen lógica de negocio

Documentación de la API

La API REST está completamente documentada mediante **Swagger UI**, accesible en <http://localhost:8080/swagger-ui.html> con la aplicación en ejecución. La documentación cubre los cinco grupos de endpoints del sistema (Autenticación, Mochilas, Categorías, Armario e Ítems de mochila), incluyendo parámetros, cuerpos de petición y códigos de respuesta de cada operación.

5. Manual técnico

Requisitos previos

Herramienta	Versión mínima
Java JDK	21
Docker Desktop	Cualquier versión reciente
Node.js	18+ (solo para el frontend)

Requisitos previos

Herramienta	Versión mínima
Java JDK	21
Maven	3.9+
MySQL	8+
Node.js	18+ (solo frontend)

Arranque del backend

1. Abrir MySQL como administrador y crear el usuario:

```
CREATE DATABASE IF NOT EXISTS backpack_db;
```

```
CREATE USER 'user'@'localhost' IDENTIFIED BY 'password';
```

```
GRANT ALL PRIVILEGES ON backpack_db.* TO 'user'@'localhost';
```

```
FLUSH PRIVILEGES;
```

2. Arrancar el backend:

```
mvn spring-boot:run
```

- Backend: <http://localhost:8080>
- Swagger UI: <http://localhost:8080/swagger-ui.html>
- Hibernate crea las tablas automáticamente al arrancar.

Arranque del frontend

cd frontend

npm install

npm run dev

- Frontend: <http://localhost:5173>

6. Informe de pruebas

Herramientas utilizadas

- **JUnit 5** — framework de pruebas unitarias
- **Mockito** — simulación de dependencias (repositorios y encoders)
- Las pruebas son unitarias puras: los repositorios se mockean, no se accede a base de datos real

Resumen de coberturas

Clase bajo prueba	Nº de tests
UsuarioService	9
MochilaService	13
CategoriaService	9
ItemArmarioService	10
ItemMochilaService	12
Arranque del contexto	1
Total	54